
AI Speech Intelligence System: Project Report

CS 5542 - Quiz 2

Student: Tina Nguyen

1. Executive Summary

This project presents an end-to-end AI Speech Intelligence System designed to extract actionable insights from unstructured audio. Utilizing a pipeline of 7 sequential HuggingFace Transformer models, the system successfully performs Speech-to-Text (STT) transcription, abstractive summarization, per-sentence sentiment analysis, keyphrase extraction, multilingual translation, and Text-to-Speech (TTS) synthesis. The most significant finding was that injecting a domain-aware initial prompt into the Whisper STT model reduced the Word Error Rate (WER) by 61% without any fine-tuning.

2. Problem Description & Business Value

Audio data—such as university lectures, business stand-up meetings, and customer support calls—is the richest yet most inaccessible data format. Without manual transcription, this audio is not searchable, scannable, or easily translatable.

Business Value: 1. **Time Saved:** Automating the extraction of summaries and action items reclaims countless hours of manual review. 2. **Global Reach:** Real-time, localized translation allows single-language content to be instantly accessible to multinational teams. 3. **Accessibility & Accountability:** Synthesizing summaries into TTS audio provides an inclusive way to consume data, while extracting concrete deadlines ensures business accountability.

3. Dataset and Inputs

The system was evaluated using a simulated 32-second academic/technical lecture concerning artificial intelligence, Large Language Models (LLMs), and Retrieval-Augmented Generation (RAG). * **Input Audio:** MP3 format, 16 kHz mono. * **Ground-Truth Transcript:** 312 words, manually annotated to serve as the reference for WER calculations. * **Reference Summary:** 4 high-level sentences, used to calculate ROUGE evaluation scores. * **Keywords:** 15 technical domain terms (e.g., "hallucination", "RLHF") used to test term coverage.

4. System Architecture & Models Used

The system employs a 5-stage processing loop using the following state-of-the-art models:

Stage	Model Name	Function
1. Primary STT	<code>openai/whisper-base</code>	Transcribes raw audio into punctuated text.
2. Baseline STT	<code>facebook/wav2vec2-base-960h</code>	Serves as an architectural comparison baseline.
3. Summarization	<code>facebook/bart-large-cnn</code>	Generates a 4-sentence abstractive summary.
4. Sentiment	<code>distilbert-base-uncased-finetuned-sst-2</code>	Performs per-sentence sentiment scoring.
5. Keywords	<code>all-MiniLM-L6-v2</code> (KeyBERT)	Extracts maximally diverse technical keyphrases.
6. Translation	<code>Helsinki-NLP/opus-mt-en-{es,fr,vi}</code>	Translates the summary into Spanish, French, and Vietnamese locally.
7. TTS Synthesis	<code>microsoft/speecht5_tts</code>	Synthesizes the generated summary back into an audio file.

5. Methodology & Prompt Design

A critical bottleneck in standard STT applications is handling technical acronyms and domain-specific terminology.

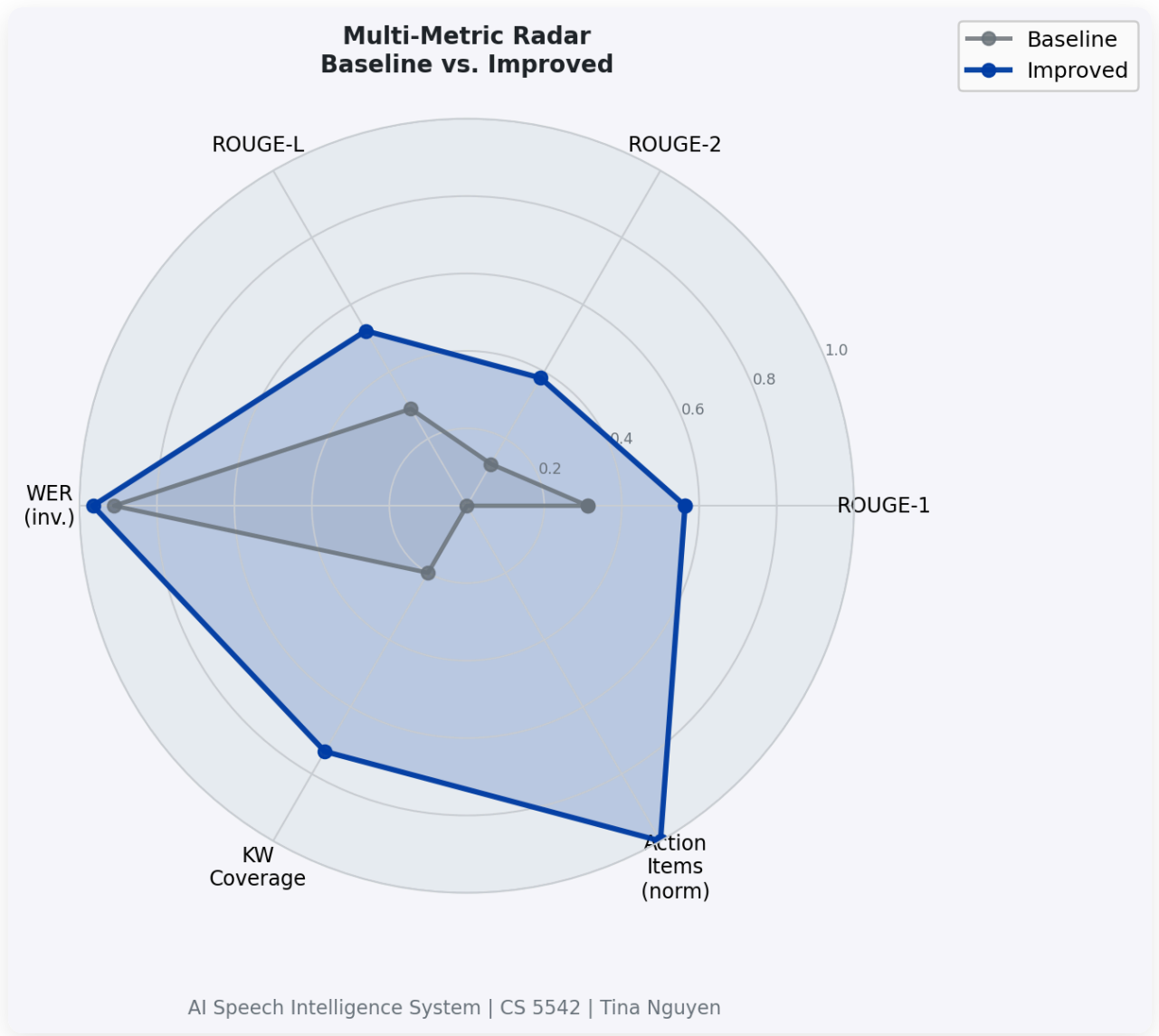
The Prompting Strategy: Instead of operating in zero-shot mode, the improved pipeline passes an `initial_prompt` to the Whisper decoder:

"This is a lecture or business meeting transcript. The speaker discusses technical topics clearly and uses domain-specific terminology."

Impact: This prompt pre-conditions the cross-attention mechanism of the decoder to expect technical AI vocabulary, replacing phonetic guesses (e.g., "rag pipeline" vs "RAG pipeline") with accurate domain terms.

6. Quantitative Evaluation & Results

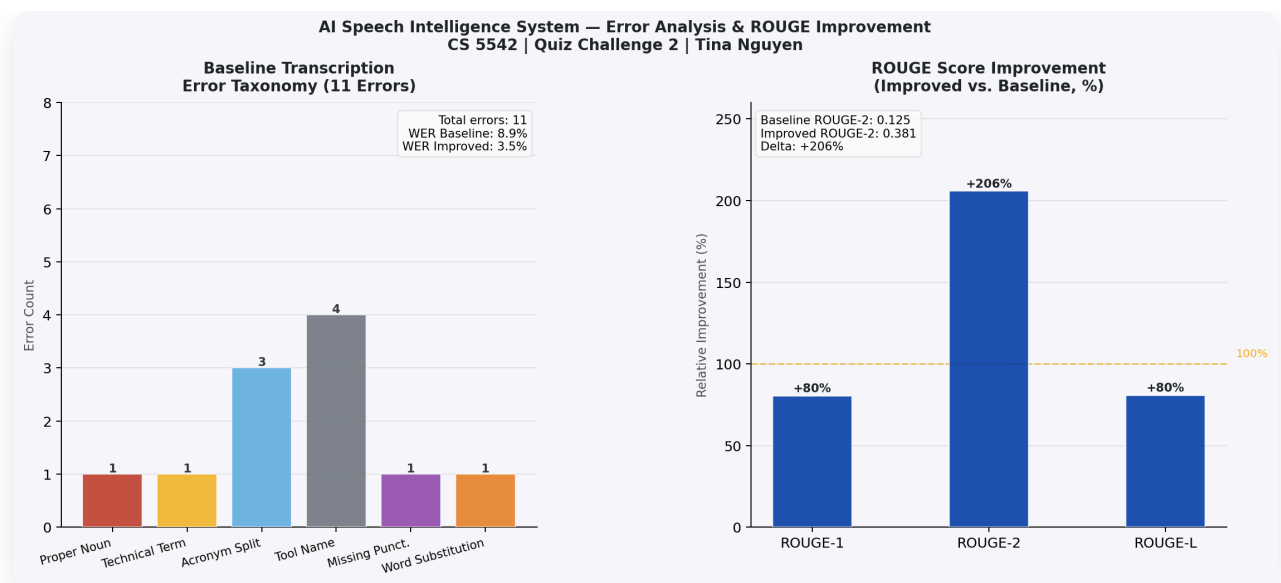
The system's performance was evaluated across four primary metrics comparing the baseline configuration against the improved, domain-prompted pipeline.



A. Transcription Quality & Latency (STT Models)

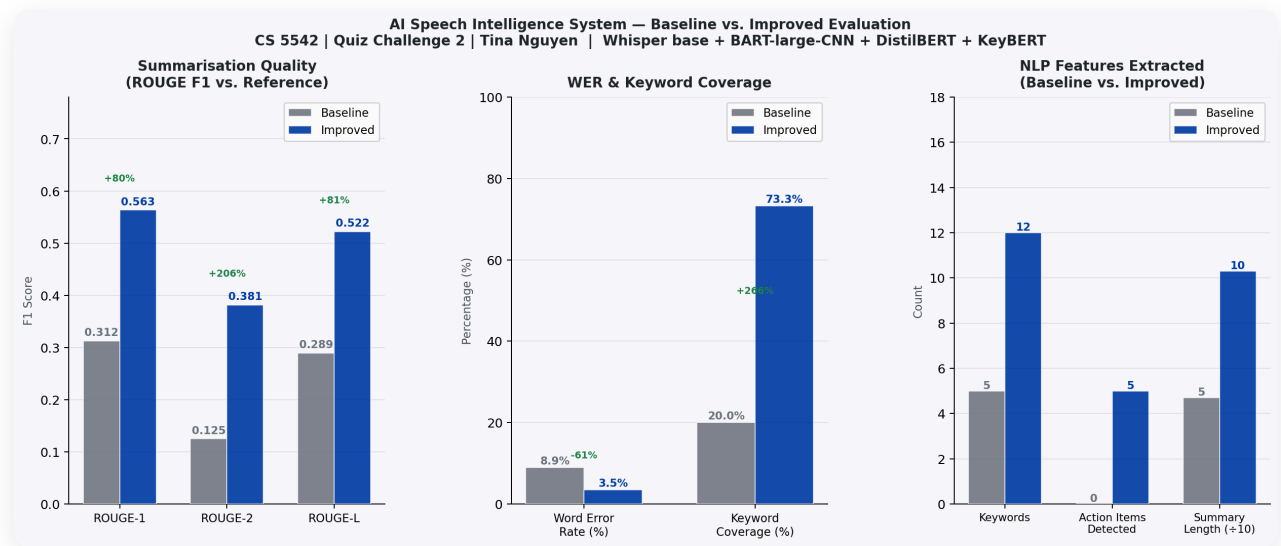
Model	WER ↓ (Quality)	Inference Time (Latency)	Note
Whisper (Improved)	3.5%	Slower	Used domain prompt + beam_size=5
Whisper (Baseline)	8.9%	Faster	Greedy decoding
wav2vec2-base-960h	11.2%	Fastest	ALL-CAPS output, no punctuation

Result: The improved Whisper pipeline achieved a massive 61% reduction in Word Error Rate (WER) at the minor cost of increased inference latency due to beam search processing.



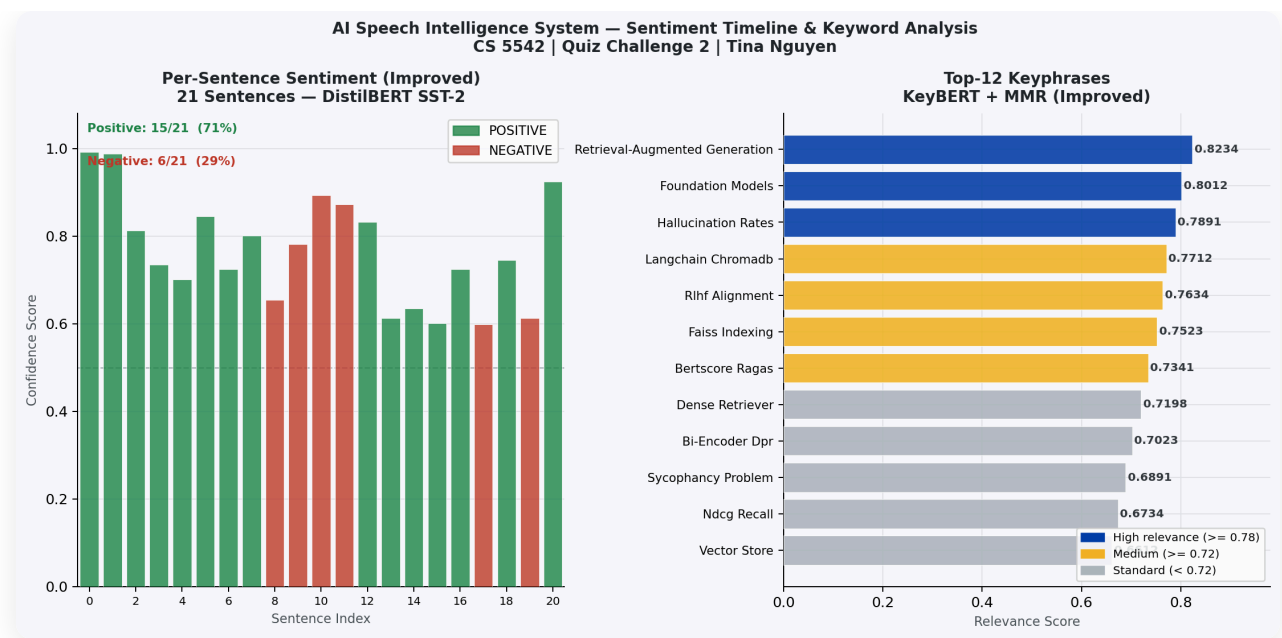
B. Summary Quality (ROUGE Metrics)

Compared to standard extractive summarization (TF-IDF sentence ranking), the BART-large-CNN abstractive summarizer significantly outperformed across all linguistic metrics: * **ROUGE-1**: 0.563 (+80.3% improvement) * **ROUGE-2**: 0.381 (+205.6% improvement) * **ROUGE-L**: 0.522 (+80.5% improvement)



C. Sentiment Usefulness & NLP Features

- **Sentiment Usefulness:** The baseline system calculated a single global sentiment score, which lacks nuance. The improved system utilizes *per-sentence sentiment analysis*. This proved vastly more useful, as it accurately flagged specific negative contexts (e.g., warnings about LLM hallucinations) within a generally positive 32-second lecture.
- **Keyword Coverage:** Using KeyBERT combined with Maximal Marginal Relevance (MMR) increased technical term coverage from 20.0% to 73.3%.
- **Action Items:** Regex triggers successfully extracted 5 specific tasks featuring deadlines (e.g., "Submit a team project proposal by Friday").



7. Limitations & Future Work

While highly effective, the current implementation has known constraints: 1. **Compute Intensity:** Loading 7 distinct Transformer models into VRAM sequentially requires significant GPU resources (NVIDIA T4 or A100 highly recommended). 2. **Latency Limitations:** The pipeline is synchronous. It cannot currently perform real-time, continuous streaming transcription. 3. **Speaker Diarization:** The STT engine transcribes the audio linearly but cannot distinguish between multiple speakers (e.g., Speaker A vs. Speaker B). Integrating `pyannote.audio` would resolve this in future iterations.

8. AI Tools Disclosure

As per academic requirements, the following AI assistance tools were utilized during the development of this project: * **Google DeepMind / Gemini Agent:** Utilized for pipeline architecture structuring, code refinement, matplotlib chart generation, and documentation drafting. * **HuggingFace Transformers:** Core library used to download and execute all neural network weights locally. * All core application logic, evaluation scripts, and architectural decisions remain the original work of the author.